Григорьев Ю.А

Методические указания к лабораторной работе 3.

Запросы PostgreSQL

1. Цель лабораторной работы 1
2. Подключение к виртуальной машине и создание БД 1
3. Запросы 2
3.1. Дополнительные возможности команды SELECT 2
3.2. Соединения 6
3.3. Агрегирование и группировка 9
3.4. Подзапросы 12
4. Контрольные вопросы и задания 16
5. Требования к отчёту 17
6. Литература 18
7. Приложение. Описание схемы базы данных и таблиц 18

Внимание! Перед выполнением лабораторной работы необходимо полностью прочитать методические указания.

1. Цель лабораторной работы

Целью лабораторной работы является формирование следующей компетенции: студент должен научиться разрабатывать сложные запросы SELECT.

2. Подключение к виртуальной машине и создание БД

1. Запустите виртуальную машину. Откройте терминал (пункт Open Terminal контекстного меню).
2. Создайте каталог lab3 в домашнем каталоге /home/user

mkdir lab3

3. Перепишите файл с описанием демонстрационной базы данных demo_small.sql из яндекс-папки /Файлы/lab3 в /home/user/lab3 своей ВМ.

4. Создайте базу данных.

а) подключитесь к Ubuntu как пользователь postgres:

```
sudo -i -u postgres
```

б) создайте базу данных с именем demo в PostgreSQL.

```
psql -f /home/user/lab3/demo_small.sql -U postgres > demo.log 2>demo.err
```

Надо немного подождать.

Примечание. Можно разделить стандартное устройство вывода и стандартное устройство вывода ошибок. Обычные сообщения будут перенаправлены в файл demo.log, а сообщения об ошибках — в файл demo.err. Обратите внимание, что между цифрой 2, обозначающей дескриптор стандартного устройства вывода сообщений об ошибках, и знаком «>», обозначающим переадресацию вывода, не должно быть пробела.

в) перейдите к базе данных demo:

```
psql -d demo -U postgres
```

г) выполните следующие команды в оболочке:

```
\dn   посмотрите список всех схем
SET search_path TO bookings;    переключитесь на схему bookings!!
\dt+  посмотрите список всех таблиц:
```

3. Запросы

Умение писать SQL-запросы — это не только ремесло, но, пожалуй, и искусство тоже. В этой л/р мы рассмотрим более сложные SQL-запросы.

Примечание. Если вы решили прервать ввод команды, еще не введя ее полностью, то просто нажмите клавиши <Ctrl>+<C>, в результате ввод команды будет прерван, а приглашение к вводу, выводимое утилитой psql, примет свой первоначальный вид.

3.1. Дополнительные возможности команды SELECT

Основой для экспериментов в этом разделе будут самые маленькие (по числу строк) таблицы базы данных «Авиаперевозки»: «Самолеты» (aircrafts) и

«Аэропорты» (airports). Прежде чем перейти к конкретным запросам, просто просмотрите содержимое этих двух таблиц. Таблица «Самолеты» совсем маленькая, а таблица «Аэропорты» содержит чуть больше ста строк.

SELECT * FROM aircrafts;

SELECT * FROM airports; потом для выхода введите q

Начнем с различных условий отбора строк в предложении WHERE. Эти условия могут конструироваться с использованием следующих операторов сравнения: =, < >, >, > =, <, < =. Рассмотрим некоторые способы осуществления отбора строк. Для начала поставим перед собой такую задачу: выбрать все самолеты компании Airbus. В этом нам поможет оператор поиска шаблонов LIKE:

SELECT * FROM aircrafts WHERE model LIKE 'Airbus%';

Обратите внимание на символ «%», имеющий специальное значение. Он соответствует любой последовательности символов, т. е. вместо него могут быть подставлены любые символы в любом количестве, а может и не быть подставлено ни одного символа. Однако в этом случае нужно учитывать следующие соображения. Если по тому столбцу, к которому применяется оператор LIKE, создан индекс для ускорения доступа к данным, то при наличии символа «%» в начале шаблона этот индекс использоваться не будет. Из-за этого может ухудшиться производительность, т. е. запрос будет выполняться медленнее.

Кроме символа «%» в шаблоне может использоваться и символ подчеркивания _ «_»,который соответствует в точности одному любому символу. В качестве примера найдем в таблице «Аэропорты» те из них, которые имеют названия длиной три символа (буквы). С этой целью зададим в качестве шаблона строку, состоящую из трех символов «_».

SELECT * FROM airports WHERE airport_name LIKE '___';

Существует ряд операторов для работы с регулярными выражениями POSIX. Эти операторы имеют больше возможностей, чем оператор LIKE. Для того чтобы выбрать, например, самолеты компаний Airbus и Boeing, можно сделать так:

SELECT * FROM aircrafts WHERE model ~ '^(A|Boe)';

Оператор ~ ищет совпадение с шаблоном с учетом регистра символов. Символ «^» в начале регулярного выражения означает, что поиск совпадения будет привязан к началу строки. Если же требуется проверить наличие такого символа в составе строки, то перед ним нужно поставить символ обратной косой черты «\». Выражение в круглых скобках означает альтернативный выбор между значениями, разделяемыми символом «|». Поэтому в выборку попадут значения, начинающиеся либо на «A», либо на «Boe».

Для инвертирования смысла оператора ~ нужно перед ним добавить знак «!». В качестве примера отыщем модели самолетов, которые не завершаются числом 300.

SELECT * FROM aircrafts WHERE model !~ '300$';

В этом регулярном выражении символ «$» означает привязку поискового шаблона к концу строки. Если же требуется проверить наличие такого символа в составе строки, то перед ним нужно поставить символ обратной косой черты «\».

Использование регулярных выражений подробно рассматривается в разделе документации 9.7.3 «Регулярные выражения POSIX».

В качестве замены традиционных операторов сравнения могут использоваться предикаты сравнения, которые ведут себя так же, как и операторы, но имеют другой синтаксис. Давайте ответим на вопрос: какие самолеты имеют дальность полета в диапазоне от 3 000 км до 6 000 км? Ответ получим с помощью предиката BETWEEN.

```
SELECT * FROM aircrafts WHERE range BETWEEN 3000 AND 6000;
```

Обратите внимание, что граничное значение 3000 включено в полученную выборку. При выборке данных можно проводить вычисления и получать в результирующей таблице вычисляемые столбцы. Если мы захотим представить дальность полета не только в километрах, но и в милях, то нужно вычислить это выражение и для удобства присвоить новому столбцу псевдоним с помощью ключевого слова AS.

```
SELECT model, range, range / 1.609 AS miles FROM aircrafts;
```

По всей вероятности, такая высокая точность представления значений в милях не требуется, поэтому мы можем уменьшить ее до разумного предела в два десятичных знака:

```
SELECT model, range, round( range / 1.609, 2 ) AS miles FROM aircrafts;
```

Теперь обратимся к такому вопросу, как упорядочение строк при выводе. Если не принять специальных мер, то СУБД не гарантирует никакого конкретного порядка строк в результирующей выборке. Для упорядочения строк служит предложение ORDER BY. Например, если мы захотим разместить самолеты в порядке убывания дальности их полета, то нужно сделать так:

```
SELECT * FROM aircrafts ORDER BY range DESC;
```

Мы детально разобрались с таблицей «Самолеты» и теперь обратим наше внимание на таблицу «Аэропорты». В ней есть столбец «Часовой пояс» (timezone). Давайте посмотрим, в каких различных часовых поясах располагаются аэропорты. Если сделать традиционную выборку

```
SELECT timezone FROM airports;
```

то мы получим список значений, среди которых будет много повторяющихся. Конечно, это неудобно. Для того чтобы оставить в выборке только неповторяющиеся значения, служит ключевое слово DISTINCT:

```
SELECT DISTINCT timezone FROM airports ORDER BY 1;
```

Обратите внимание, что столбец, по значениям которого будут упорядочены строки, указан не с помощью его имени, а с помощью его порядкового номера в предложении SELECT.

Таким образом, аэропорты располагаются в семнадцати различных часовых поясах. Они описаны в базе данных часовых поясов, поддерживаемой международной организацией IANA (Internet Assigned Numbers Authority), и отличаются от традиционных географических и административных часовых поясов, число которых в России равно одиннадцати.

В таблице «Аэропорты» более ста записей. Если мы поставим задачу найти три самых восточных аэропорта, то для ее решения подошел бы такой запрос

```
SELECT airport_name, city, longitude
FROM airports
ORDER BY longitude DESC
LIMIT 3;
```

А как найти еще три аэропорта, которые находятся немного западнее первой тройки, т. е. занимают места с четвертого по шестое? Для пропуска строк служит предложение OFFSET:

```
SELECT airport_name, city, longitude
FROM airports
ORDER BY longitude DESC
LIMIT 3
OFFSET 3;
```

В дополнение к вычисляемым столбцам, когда выводимые значения получают путем вычислений, при выборке данных из таблиц можно использовать условные выражения, позволяющие вывести то или иное значение в зависимости от условий.

В таблице «Самолеты» есть столбец «Максимальная дальность полета» (range). Мы можем дополнить вывод данных из этой таблицы столбцом «Класс самолета», имея в виду принадлежность каждого самолета к классу дальнемагистральных, среднемагистральных или ближнемагистральных судов. Для этого подойдет конструкция

```
CASE WHEN условие THEN выражение
[ WHEN ... ]
```

[ELSE выражение]
END

Воспользовавшись этой конструкцией в предложении SELECT и назначив новому столбцу имя с помощью ключевого слова AS, получим следующий запрос:

```
SELECT model, range,
CASE WHEN range < 2000 THEN 'Ближнемагистральный'
WHEN range < 5000 THEN 'Среднемагистральный'
ELSE 'Дальнемагистральный'
END AS type
FROM aircrafts
ORDER BY model;
```

3.2. Соединения

В тех случаях, когда информации, содержащейся в одной таблице, недостаточно для получения требуемого результата, используют соединение (join) таблиц. Покажем способ выполнения соединения на примере следующего запроса: выбрать все места, предусмотренные компоновкой салона самолета Cessna 208 Caravan:

```
SELECT a.aircraft_code, a.model, s.seat_no, s.fare_conditions
FROM seats AS s
JOIN aircrafts AS a
ON s.aircraft_code = a.aircraft_code
WHERE a.model ~ '^Cessna'
ORDER BY s.seat_no;
```

В соединении одна и та же таблица может участвовать дважды, т. е. формировать соединение таблицы с самой собой. В качестве примера рассмотрим запрос для создания представления «Рейсы» (flights_v). Этот запрос выглядит так:

```
CREATE OR REPLACE VIEW flights_v AS
SELECT f.flight_id,
f.flight_no,
f.scheduled_departure,
timezone( dep.timezone, f.scheduled_departure )
AS scheduled_departure_local,
f.scheduled_arrival,
timezone( arr.timezone, f.scheduled_arrival )
AS scheduled_arrival_local,
f.scheduled_arrival - f.scheduled_departure
AS scheduled_duration,
```

```
f.departure_airport,
dep.airport_name AS departure_airport_name,
dep.city AS departure_city,
f.arrival_airport,
arr.airport_name AS arrival_airport_name,
arr.city AS arrival_city,
f.status,
f.aircraft_code,
f.actual_departure,
timezone( dep.timezone, f.actual_departure )
AS actual_departure_local,
f.actual_arrival,
timezone( arr.timezone, f.actual_arrival )
AS actual_arrival_local,
f.actual_arrival - f.actual_departure AS actual_duration
FROM flights f,
airports dep,
airports arr
WHERE f.departure_airport = dep.airport_code
AND f.arrival_airport = arr.airport_code;
```

В этом представлении используется не только таблица «Рейсы» (flights), но также и таблица «Аэропорты» (airports). Причем она используется, условно говоря, дважды. Это позволяет учесть, что аэропорты вылета и назначения не совпадают.

Теперь обратимся к так называемым внешним соединениям. Зададимся вопросом: сколько маршрутов обслуживают самолеты каждого типа? Если не требовать вывода наименований моделей самолетов, тогда всю необходимую информацию можно получить из материализованного представления «Маршруты» (routes). Но мы все же будем выводить и наименования моделей, поэтому обратимся также к таблице «Самолеты» (aircrafts). Соединим эти таблицы на основе атрибута aircraft_code, сгруппируем строки и просто воспользуемся функцией count. В этом запросе внешнее соединение еще не используется.

```
SELECT r.aircraft_code, a.model, count( * ) AS num_routes
FROM routes r
JOIN aircrafts a ON r.aircraft_code = a.aircraft_code
GROUP BY 1, 2
ORDER BY 3 DESC;
```

Обратите внимание, что таблица «Самолеты» содержит 9 моделей, а в этой выборке лишь 8 строк. Значит, какая-то модель самолета не участвует в выполнении рейсов. Как ее выявить? С помощью такого запроса:

```
SELECT a.aircraft_code AS a_code, a.model, r.aircraft_code AS r_code,
count( r.aircraft_code ) AS num_routes
FROM aircrafts a
LEFT OUTER JOIN routes r ON r.aircraft_code = a.aircraft_code
GROUP BY 1, 2, 3
ORDER BY 4 DESC;
```

В данном запросе используется левое внешнее соединение — об этом говорит предложение LEFT OUTER JOIN. В качестве базовой таблицы выбирается таблица aircrafts, указанная в запросе слева от предложения LEFT OUTER JOIN, и для каждой строки, находящейся в ней, из таблицы routes подбираются строки, в которых значение атрибута aircraft_code такое же, как и в текущей строке таблицы aircrafts. Если для строки из левой таблицы (левой относительно предложения LEFT OUTER JOIN) не находится ни одной соответствующей строки в правой таблице, тогда в результирующую строку вместо значений столбцов правой таблицы будут помещены значения NULL.

Кроме левого внешнего соединения существует также и правое внешнее соединение — RIGHT OUTER JOIN. В этом случае в качестве базовой выбирается таблица, имя которой указано справа от предложения RIGHT OUTER JOIN. Комбинацией этих двух видов внешних соединений является полное внешнее соединение — FULL OUTER JOIN. В этом случае в выборку включаются строки из левой таблицы, для которых не нашлось соответствующих строк в правой таблице, и строки из правой таблицы, для которых не нашлось соответствующих строк в левой таблице.

В практической работе при выполнении выборок зачастую выполняются многотабличные запросы, включающие три таблицы и более. В качестве примера рассмотрим такую задачу: определить число пассажиров, не пришедших на регистрацию билетов и, следовательно, не вылетевших в пункт назначения. Будем учитывать только рейсы, у которых фактическое время вылета не пустое, т. е. рейсы, имеющие статус Departed или Arrived.

```
SELECT count( * )
FROM ( ticket_flights t
JOIN flights f ON t.flight_id = f.flight_id
)
LEFT OUTER JOIN boarding_passes b
ON t.ticket_no = b.ticket_no AND t.flight_id = b.flight_id
WHERE f.actual_departure IS NOT NULL AND b.flight_id IS NULL;
```

Оказывается, таких пассажиров нет.

В команде SELECT предусмотрены средства для выполнения операций с выборками, как с множествами, а именно:

– UNION для вычисления объединения множеств строк из двух выборок;

– INTERSECT для вычисления пересечения множеств строк из двух выборок;

– EXCEPT для вычисления разности множеств строк из двух выборок.

Запросы должны возвращать одинаковое число столбцов, типы данных у столбцов также должны совпадать.

Вопрос: в какие города можно улететь либо из Москвы, либо из Санкт-Петербурга?

```
SELECT arrival_city FROM routes
WHERE departure_city = 'Москва'
UNION
SELECT arrival_city FROM routes
WHERE departure_city = 'Санкт-Петербург'
ORDER BY arrival_city;
```

Строки-дубликаты в результирующее множество не включаются. Для их включения нужно использовать UNION ALL.

3.3. Агрегирование и группировка

Среди множества функций, имеющихся в PostgreSQL, важное место занимают агрегатные функции. С одной из них, функцией count, мы уже работали довольно много. Давайте рассмотрим еще ряд функций из этой группы и сделаем это на примере таблицы «Бронирования».

Для расчета среднего значения по столбцу используется функция avg (от слова average).

```
SELECT avg( total_amount ) FROM bookings;
```

Для получения максимального (или минимального) значения по столбцу используется функция max (или min).

Рассмотрим два примера. Первый будет таким: давайте подсчитаем, сколько маршрутов предусмотрено из Москвы в другие города. При формировании запроса не будем учитывать частоту рейсов в неделю, т. е. независимо от того, выполняется какой-то рейс один раз в неделю или семь раз, он учитывается только однократно. Воспользуемся материализованным представлением «Маршруты».

```
SELECT arrival_city, count( * )
FROM routes
WHERE departure_city = 'Москва'
GROUP BY arrival_city
ORDER BY count DESC;
```

В качестве второго примера рассмотрим ситуацию, когда руководству компании потребовалась обобщенная информация по частоте выполнения рейсов, а именно: сколько рейсов выполняется ежедневно, сколько рейсов — шесть дней в неделю, пять и т. д. Опять обратимся к материализованному представлению «Маршруты». Но теперь при формировании запроса, в отличие от первого примера, воспользуемся столбцом days_of_week, в котором содержатся массивы номеров дней недели, когда выполняется данный рейс.

```
SELECT array_length( days_of_week, 1 ) AS days_per_week,
count( * ) AS num_routes
FROM routes
GROUP BY days_per_week
ORDER BY 1 desc;
```

В этом запросе используется функция array_length, возвращающая количество элементов в указанном измерении массива. Поскольку массив одномерный, то вторым параметром функции будет число 1 — первое измерение.

Можно задать в предложении HAVING. Важно помнить, что предложение WHERE работает с отдельными строками еще до выполнения группировки с помощью GROUP BY, а предложение HAVING — уже после выполнения группировки.

В качестве примера приведем такой запрос: определить, сколько существует маршрутов из каждого города в другие города, и вывести названия городов, из которых в другие города существует не менее 15 маршрутов.

```
SELECT departure_city, count( * )
FROM routes
GROUP BY departure_city
HAVING count( * ) >= 15
ORDER BY count DESC;
```

Кроме обычных агрегатных функций существуют и так называемые оконные функции (window functions), технология использования которых описана в документации в разделе 3.5 «Оконные функции». Эти функции предоставляют возможность производить вычисления на множестве строк, логически связанных с текущей строкой, т. е. имеющих то или иное отношение к ней. При работе с оконными функциями используются концепции раздела (partition) и оконного кадра (window frame). Сначала объясним эти понятия на примере.

Предположим, что руководство нашей компании хочет усовершенствовать тарифную политику и с этой целью просит нас предоставить сведения о распределении количества проданных билетов на

некоторые рейсы во времени. Количество проданных билетов должно выводиться в виде накопленного показателя, суммирование должно производиться в пределах каждого календарного месяца.

Более детально, в столбцах book_ref и book_date выборки должны приводиться номер и время бронирования соответственно. В столбцах month и day должны указываться порядковый номер месяца и день этого месяца. Столбец count должен содержать суммарные (накопленные) количества билетов, проданных на каждый момент времени. С первого дня нового месяца подсчет числа проданных билетов начинается сначала.

```
SELECT b.book_ref,
b.book_date,
extract( 'month' from b.book_date ) AS month,
extract( 'day' from b.book_date ) AS day,
count( * ) OVER (
PARTITION BY date_trunc( 'month', b.book_date )
ORDER BY b.book_date
) AS count
FROM ticket_flights tf
JOIN tickets t ON tf.ticket_no = t.ticket_no
JOIN bookings b ON t.book_ref = b.book_ref
WHERE tf.flight_id = 1
ORDER BY b.book_date;
```

Рассмотрим конструкцию, предназначенную для вызова оконной функции:

```
count( * ) OVER (
PARTITION BY date_trunc( 'month', b.book_date )
ORDER BY b.book_date
) AS count
```

В этой конструкции обязательным является ключевое слово OVER. Функция count — это обычная агрегатная функция, но если вслед за ней идет это ключевое слово, то она становится оконной функцией. Предложение PARTITION BY задает правило разбиения строк выборки на разделы. Предложение ORDER BY предписывает порядок сортировки строк в разделах.

Оконные функции, в отличие от обычных агрегатных функций, не требуют группировки строк, а работают на уровне отдельных (несгруппированных) строк.

Более подробно использование оконных функций описано в документации. Мы рекомендуем начать с раздела 3.5 «Оконные функции», в котором приводятся примеры их использования. В разделе 9.21 «Оконные функции» приводятся описания всех оконных функций, предлагаемых PostgreSQL. В разделе 4.2.8 «Вызовы оконных функций» детально рассматривается синтаксис вызова оконных функций. В разделе 7.2.5

«Обработка оконных функций» говорится о том, на каком этапе выполнения запроса производится обработка этих функций.

3.4. Подзапросы

Предположим, что сотрудникам аналитического отдела потребовалось провести статистическое исследование финансовых результатов работы авиакомпании. В качестве первого шага они решили подсчитать количество операций бронирования, в которых общая сумма превышает среднюю величину по всей выборке.

```
SELECT count( * ) FROM bookings
WHERE total_amount >
( SELECT avg( total_amount ) FROM bookings );
```

Подзапросы могут присутствовать в предложениях SELECT, FROM, WHERE и HAVING, а также в предложении WITH, о котором мы расскажем позднее.

В качестве примера давайте выясним, какие маршруты существуют между городами часового пояса Asia/Krasnoyarsk. Подзапрос будет выдавать список городов из этого часового пояса, а в предложении WHERE главного запроса с помощью предиката IN будет выполняться проверка на принадлежность города этому списку. При этом подзапрос выполняется только один раз для всего внешнего запроса, а не при обработке каждой строки из таблицы routes во внешнем запросе. Повторного выполнения подзапроса не требуется, т. к. его результат не зависит от значений, хранящихся в таблице routes. Такие подзапросы называются некоррелированными.

```
SELECT flight_no, departure_city, arrival_city
FROM routes
WHERE departure_city IN (
SELECT city
FROM airports
WHERE timezone ~ 'Krasnoyarsk'
)
AND arrival_city IN (
SELECT city
FROM airports
WHERE timezone ~ 'Krasnoyarsk'
);
```

Иногда возникают ситуации, когда от подзапроса требуется лишь установить сам факт наличия или отсутствия строк в конкретной таблице, удовлетворяющих определенному условию, а непосредственные значения

атрибутов в этих строках интереса не представляют. В подобных случаях используют предикат EXISTS (или NOT EXISTS). В качестве примера выясним, в какие города нет рейсов из Москвы.

```
SELECT DISTINCT a.city
FROM airports a
WHERE NOT EXISTS (
SELECT * FROM routes r
WHERE r.departure_city = 'Москва'
AND r.arrival_city = a.city
)
AND a.city <> 'Москва'
ORDER BY city;
```

В качестве еще одного примера использования подзапроса в предложении FROM решим такую задачу: получить перечень аэропортов в тех городах, в которых больше одного аэропорта.

```
SELECT aa.city, aa.airport_code, aa.airport_name
FROM (
SELECT city, count( * )
FROM airports
GROUP BY city
HAVING count( * ) > 1
) AS a
JOIN airports AS aa ON a.city = aa.city
ORDER BY aa.city, aa.airport_name;
```

Для иллюстрации использования подзапросов в предложении HAVING решим такую задачу: определить число маршрутов, исходящих из тех аэропортов, которые расположены восточнее географической долготы 150◦.

```
SELECT departure_airport, departure_city, count( * )
FROM routes
GROUP BY departure_airport, departure_city
HAVING departure_airport IN (
SELECT airport_code
FROM airports
WHERE longitude > 150
)
ORDER BY count DESC;
```

В сложных запросах могут использоваться вложенные подзапросы. Это означает, что один подзапрос находится внутри другого. Давайте в качестве примера рассмотрим такую ситуацию: руководство авиакомпании хочет

выяснить степень заполнения самолетов на всех рейсах, ведь отправлять полупустые самолеты не очень выгодно. Таким образом, запрос должен не только выдавать число билетов, проданных на данный рейс, и общее число мест в самолете, но должен также вычислять отношение этих двух показателей. Вот какой запрос получился:

```
SELECT ts.flight_id,
ts.flight_no,
ts.scheduled_departure_local,
ts.departure_city,
ts.arrival_city,
a.model,
ts.fact_passengers,
ts.total_seats,
round( ts.fact_passengers::numeric /
ts.total_seats::numeric, 2 ) AS fraction
FROM (
SELECT f.flight_id,
f.flight_no,
f.scheduled_departure_local,
f.departure_city,
f.arrival_city,
f.aircraft_code,
count( tf.ticket_no ) AS fact_passengers,
( SELECT count( s.seat_no )
FROM seats s
WHERE s.aircraft_code = f.aircraft_code
) AS total_seats
FROM flights_v f
JOIN ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.status = 'Arrived'
GROUP BY 1, 2, 3, 4, 5, 6
) AS ts
JOIN aircrafts AS a ON ts.aircraft_code = a.aircraft_code
ORDER BY ts.scheduled_departure_local;
```

Самый внутренний подзапрос — total_seats — выдает общее число мест в самолете. Этот подзапрос — коррелированный, т. к. он выполняется для каждой строки, обрабатываемой во внешнем подзапросе, т. е. для каждой модели самолета. Для подсчета числа проданных билетов мы использовали соединение представления «Рейсы» (flights_v) с таблицей «Перелеты» (ticket_flights) с последующей группировкой строк и вызовом функции count.

Рассмотренный сложный запрос можно сделать более наглядным за счет выделения подзапроса в отдельную конструкцию, которая называется общее табличное выражение (Common Table Expression — CTE).

```
WITH ts AS
( SELECT f.flight_id,
f.flight_no,
f.scheduled_departure_local,
f.departure_city,
f.arrival_city,
f.aircraft_code,
count( tf.ticket_no ) AS fact_passengers,
( SELECT count( s.seat_no )
FROM seats s
WHERE s.aircraft_code = f.aircraft_code
) AS total_seats
FROM flights_v f
JOIN ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.status = 'Arrived'
GROUP BY 1, 2, 3, 4, 5, 6
)
SELECT ts.flight_id,
ts.flight_no,
ts.scheduled_departure_local,
ts.departure_city,
ts.arrival_city,
a.model,
ts.fact_passengers,
ts.total_seats,
round( ts.fact_passengers::numeric /
ts.total_seats::numeric, 2 ) AS fraction
FROM ts
JOIN aircrafts AS a ON ts.aircraft_code = a.aircraft_code
ORDER BY ts.scheduled_departure_local;
```

Конструкция WITH ts AS (...) и представляет собой общее табличное выражение (CTE). Такие конструкции удобны тем, что позволяют упростить основной запрос, сделать его менее громоздким. В общем табличном выражении может присутствовать больше одного подзапроса. Каждый подзапрос формирует временную таблицу с указанным именем. Если имена столбцов этой таблицы не заданы явным образом в виде списка, тогда они определяются на основе списка столбцов в предложении SELECT. В нашем примере это будет именно так. Теперь мы можем в главном запросе обращаться к временной таблице ts так, как если бы она существовала

постоянно. Но важно учитывать, что временная таблица, создаваемая в общем табличном выражении, существует только во время выполнения запроса.

4. Контрольные вопросы и задания

1. В документации сказано, что служебный символ «%» в шаблоне оператора LIKE соответствует любой последовательности символов, в том числе и пустой последовательности, однако ничего не сказано насчет правил обработки пробелов. В таблице «Билеты» (tickets) столбец passenger_name содержит имя и фамилию пассажира, записанные заглавными латинскими буквами и разделенные одним пробелом. Выясните правила обработки пробелов самостоятельно, выполнив следующие команды и сравнив полученные результаты:

```
SELECT count( * ) FROM tickets;
SELECT count( * ) FROM tickets WHERE passenger_name LIKE '% %';
SELECT count( * ) FROM tickets WHERE passenger_name LIKE '% % %';
SELECT count( * ) FROM tickets WHERE passenger_name LIKE '% %%';
```

2. Этот запрос выбирает из таблицы «Билеты» (tickets) всех пассажиров с именами, состоящими из трех букв (в шаблоне присутствуют три символа «_»):

```
SELECT passenger_name
FROM tickets
WHERE passenger_name LIKE '___ %';
```

Предложите шаблон поиска в операторе LIKE для выбора из этой таблицы всех пассажиров с фамилиями, состоящими из пяти букв.

3. В разделе документации 9.7.2 «Регулярные выражения SIMILAR TO» рассматривается оператор SIMILAR TO. Он работает аналогично оператору LIKE, но использует шаблоны, соответствующие определению регулярных выражений, приведенному в стандарте SQL. Регулярные выражения SQL представляют собой комбинацию синтаксиса LIKE с синтаксисом обычных регулярных выражений. Самостоятельно ознакомьтесь с оператором SIMILAR TO.

4. В разделе документации 9.2 «Функция и операторы сравнения» представлены различные предикаты сравнения, кроме предиката BETWEEN, рассмотренного в этой главе. Самостоятельно ознакомьтесь с ними.

5. В разделе документации 9.17 «Условные выражения» представлены условные выражения, которые поддерживаются в PostgreSQL. В тексте главы была рассмотрена конструкция CASE. Самостоятельно ознакомьтесь с функциями COALESCE, NULLIF, GREATEST и LEAST.

6. Выясните, на каких маршрутах используются самолеты компании Boeing. В выборке вместо кода модели должно выводиться ее наименование, например, вместо кода 733 должно быть Boeing 737-300.

Указание: можно воспользоваться соединением представления «Маршруты» (routes) и таблицы «Самолеты» (aircrafts).

7. Самые крупные самолеты в нашей авиакомпании — это Boeing 777-300. Выяснить, между какими парами городов они летают. Каждая пара городов была выведена только один раз.

8. Сколько рейсов выполняется из Москвы в Санкт-Петербург? Получить результат в следующем виде: departure_city, arrival_city, count.

9. Выяснить, сколько различных рейсов выполняется из каждого города, без учета частоты рейсов в неделю, можно с помощью обращения к представлению routes (маршруты).

```
SELECT departure_city, count( * )
FROM routes
GROUP BY departure_city
ORDER BY count DESC;
```

10. Модифицируйте этот запрос так, чтобы он выводил число направлений, по которым летают самолеты из каждого города. Например, из Москвы в СанктПетербург летает несколько различных рейсов, но все эти рейсы относятся к одному направлению.

Указание: нужно передать параметр в функцию count.

11. В материализованном представлении «Маршруты» (routes) имеется столбец days_of_week, который содержит списки (массивы) номеров дней недели, когда выполняется каждый рейс. Для оптимизации расписания вылетов из Москвы нужно выявить пять городов, в которые из столицы отправляется наибольшее число ежедневных рейсов (маршрутов). Строки в выборке следует расположить в убывающем порядке числа выполняемых рейсов.

Указание: воспользуйтесь функцией array_length.

12.* Предположим, что служба материального снабжения нашей авиакомпании запросила информацию о числе рейсов, выполняющихся из Москвы в каждый день недели.

Примечание. Используйте функцию unnest.

13. Каковы максимальные и минимальные цены билетов на все направления. Оператор SELECT должен возвращать departure_city, arrival_city, max(amount), min(amount)

Примечание. Используйте представление flights_v и таблицу ticket_flights

5. Требования к демонстрации и к отчёту

1. Необходимо продемонстрировать выполнение тех запросов из разделов 3 и 4, на которые укажет преподаватель.

2. Отчет должен содержать результаты выполнения операторов и заданий, указанных в разделе «Контрольные вопросы и задания» (раздел 4).

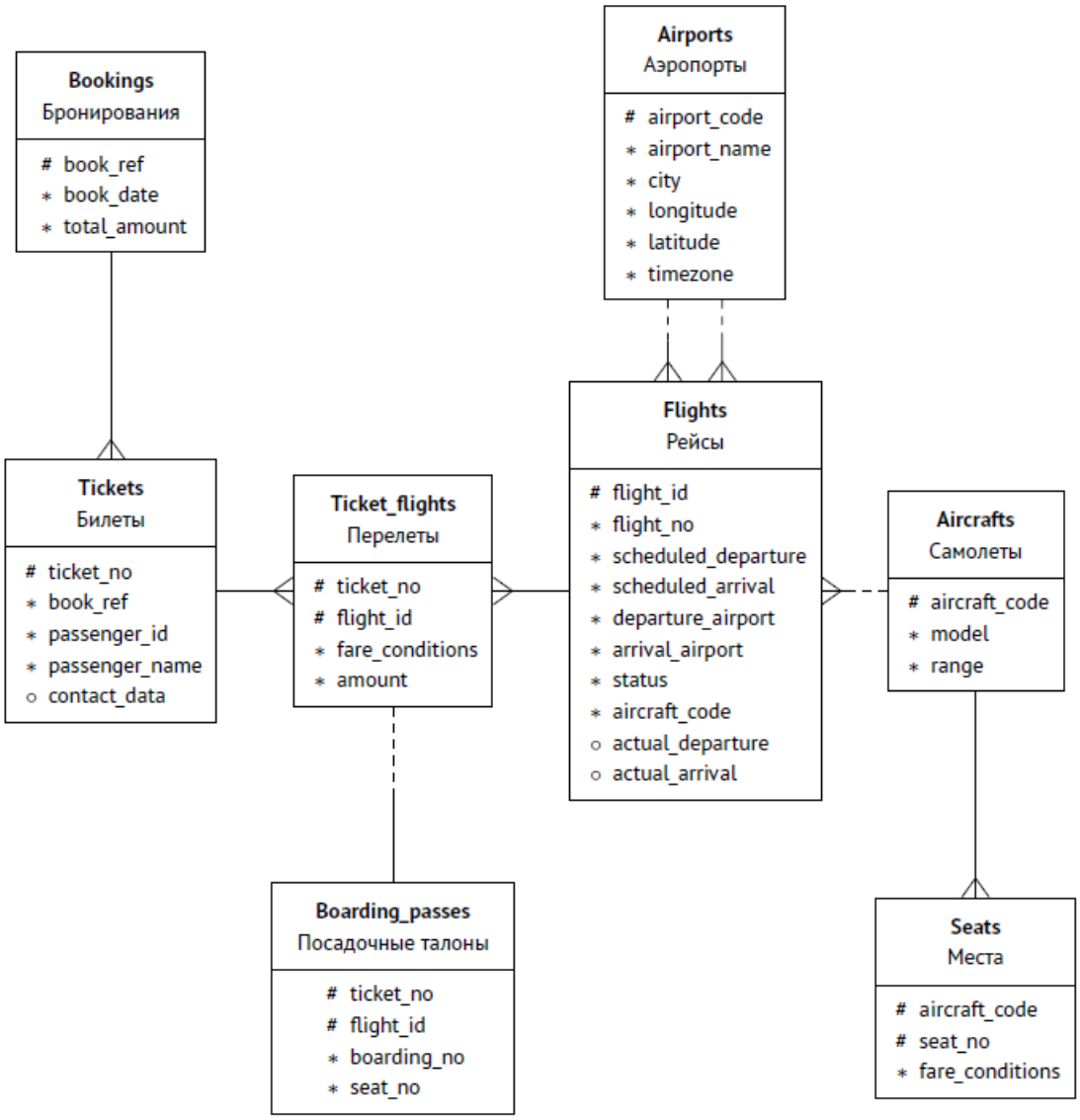
6. Литература

1. Документация по PostgreSQL 15 https://www.postgresql.org/files/documentation/pdf/15/postgresql-15-A4.pdf

2. Моргунов, Е. П. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. — СПб.: БХВ-Петербург, 2018. — 336 с.: ил.

7. Приложение. Описание схемы базы данных и таблиц.

1. Схема базы данных



2. Таблицы и представления

\d+

```
Схема   |        Имя         |            Тип             | Владелец |   Размер   |     Описание
---------+--------------------+----------------------------+----------+------------+-----------
--------
 bookings | aircrafts          | таблица                    | postgres | 16 kB      | Самолеты
 bookings | airports           | таблица                    | postgres | 48 kB      | Аэропорты
 bookings | boarding_passes    | таблица                    | postgres | 33 MB      | Посадочные
талоны
 bookings | bookings           | таблица                    | postgres | 13 MB      | Бронирования
 bookings | flights            | таблица                    | postgres | 3160 kB    | Рейсы
 bookings | flights_flight_id_seq | последовательность       | postgres | 8192 bytes |
 bookings | flights_v          | представление              | postgres | 0 bytes    | Рейсы
 bookings | routes             | материализованное представление | postgres | 144 kB     |
Маршруты
 bookings | seats              | таблица                    | postgres | 88 kB      | Места
 bookings | ticket_flights     | таблица                    | postgres | 68 MB      | Перелеты
 bookings | tickets            | таблица                    | postgres | 48 MB      | Билеты
```

\d routes

```
 Материализованное представление "bookings.routes" маршруты
      Столбец       |     Тип     | Модификаторы
---------------------+-------------+--------------
 flight_no           | character(6) |  номер полёта
 departure_airport   | character(3) | ИД аэропорта вылета
 departure_airport_name | text      | аэропорт вылета
 departure_city      | text        |  город вылета
 arrival_airport     | character(3) | ИД аэропорта прилёта
 arrival_airport_name | text       | аэропорт прилёта
 arrival_city        | text        | город прилёта
 aircraft_code       | character(3) | код самолёта
 duration            | interval    | продолжительность
 days_of_week        | integer[]   | день недели
```

\d aircrafts

```
      Таблица "bookings.aircrafts" самолёты
   Столбец   |     Тип     | Модификаторы
-------------+-------------+--------------
 aircraft_code | character(3) | NOT NULL код
 model       | text        | NOT NULL модель
 range       | integer     | NOT NULL дальность полёта
```

\d airports

Таблица "bookings.airports" аэропорты

```
  Столбец    |       Тип        | Модификаторы
-------------+------------------+-------------
 airport_code | character(3)     | NOT NULL ИД аэропорта
 airport_name | text             | NOT NULL имя аэропорта
 city         | text             | NOT NULL город
 longitude    | double precision | NOT NULL долгота
 latitude     | double precision | NOT NULL широта
 timezone     | text             | NOT NULL часовой пояс
```

\d boarding_passes

```
Таблица "bookings.boarding_passes" Посадочные талоны
  Столбец   |       Тип          | Модификаторы
------------+--------------------+-------------
 ticket_no  | character(13)      | NOT NULL номер билета
 flight_id  | integer            | NOT NULL ИД рейса
 boarding_no | integer           | NOT NULL номер посадочного талона
 seat_no    | character varying(4) | NOT NULL номер места
```

\d bookings

```
Таблица "bookings.bookings" Бронирования
  Столбец    |          Тип           | Модификаторы
-------------+------------------------+-------------
 book_ref    | character(6)           | NOT NULL ссылка
 book_date   | timestamp with time zone | NOT NULL дата
 total_amount | numeric(10,2)         | NOT NULL стоимость
```

\d flights

```
Таблица "bookings.flights" Рейсы
     Столбец      |          Тип           |                     Модификаторы
------------------+------------------------+-----------------------------------------------------
 flight_id        | integer                | NOT NULL DEFAULT nextval('flights_flight_id_seq'::regclass)
ИД рейса
 flight_no        | character(6)           | NOT NULL номер рейса
 scheduled_departure | timestamp with time zone | NOT NULL запланированный вылет
 scheduled_arrival   | timestamp with time zone | NOT NULL запланированный прилёт
 departure_airport   | character(3)       | NOT NULL аэропорт вылета
 arrival_airport     | character(3)       | NOT NULL аэропорт прилёта
 status              | character varying(20) | NOT NULL состояние рейса
 aircraft_code       | character(3)       | NOT NULL код самолёта
 actual_departure    | timestamp with time zone | время фактического вылета
 actual_arrival      | timestamp with time zone | время фактического прилёта
```

\d flights_v

```
Представление "bookings.flights_v" Рейсы
```

```
        Столбец         |           Тип            | Модификаторы
------------------------+--------------------------+-------------
 flight_id              | integer                  | ИД рейса
 flight_no              | character(6)             | номер рейса
 scheduled_departure    | timestamp with time zone| запланированный вылет
 scheduled_departure_local | timestamp without time zone | без часового пояса
 scheduled_arrival      | timestamp with time zone    | запланированный прилёт
 scheduled_arrival_local   | timestamp without time zone | без часового пояса
 scheduled_duration     | interval                 | запланированная продолжительность
 departure_airport      | character(3)       | ИД аэропорта вылета
 departure_airport_name    | text             | аэропорт вылета
 departure_city         | text                  | город вылета
 arrival_airport        | character(3)         | ИД аэропорта прилёта
 arrival_airport_name   | text                 | аэропорт прилёта
 arrival_city           | text                 | город прилёта
 status                 | character varying(20)    | состояние рейса
 aircraft_code          | character(3)             | код самолёта
 actual_departure       | timestamp with time zone    | время фактического вылета
 actual_departure_local    | timestamp without time zone | без часового пояса
 actual_arrival         | timestamp with time zone    | время фактического прилёта
 actual_arrival_local   | timestamp without time zone | без часового пояса
 actual_duration        | interval    фактическая продолжительность полёта
                |
```

```
\d seats
```

```
Таблица "bookings.seats" Места
    Столбец     |         Тип          | Модификаторы
----------------+----------------------+-------------
 aircraft_code  | character(3)         | NOT NULL код самолёта
 seat_no        | character varying(4) | NOT NULL номер места
 fare_conditions | character varying(10) | NOT NULL условия тарифа
```

```
\d ticket_flights
```

```
Таблица "bookings.ticket_flights" Перелеты
    Столбец     |         Тип          | Модификаторы
----------------+----------------------+-------------
 ticket_no      | character(13)        | NOT NULL номер билета
 flight_id      | integer              | NOT NULL ИД рейса
 fare_conditions | character varying(10) | NOT NULL условия тарифа
 amount         | numeric(10,2)        | NOT NULL цена билета
```

```
\d tickets
```

```
Таблица "bookings.tickets" Билеты
```

```
   Столбец     |          Тип          | Модификаторы
---------------+-----------------------+--------------
 ticket_no     | character(13)         | NOT NULL номер билета
 book_ref      | character(6)          | NOT NULL ссылка на журнал
 passenger_id  | character varying(20) | NOT NULL ИД пассажира
 passenger_name | text                 | NOT NULL имя пассажира
 contact_data  | jsonb                 | контактные данные
```